CARBON[6]

AUTONOMOUS AI AGENCY FRAMEWORK

Technical Architecture Specification

Version 1.0.0

INSTITUTIONAL CONFIDENTIAL

Document Date: February 2026
Classification: Enterprise Architecture Documentation

Table of Contents

I. EXECUTIVE OVERVIEW

1.1 Purpose and Scope

1.2 Architectural Philosophy

1.3 System Components Overview

II. CORE DECISION SUBSYSTEM

2.1 MDP Specification (mdp_spec.yaml)

2.2 Policy Definition (policy_definition.py)

2.3 Reward Model (reward_model.py)

III. ENHANCED MEMORY PROTOCOL (EMP)

3.1 Memory Schema (memory_schema.json)

3.2 Memory Encoder (memory_encoder.py)

3.3 Salience Engine (memory_salience_engine.py)

3.4 Retrieval Policy (memory_retrieval_policy.py)

3.5 Decay and Compression (memory_decay_and_compression.py)

3.6 Immutable Registry (immutable_memory_registry.yaml)

IV. ENVIRONMENT MODELING

4.1 State Model (environment_state_model.py)

4.2 Variable Registry (environment_variable_registry.yaml)

4.3 Observer (environment_observer.py)

4.4 Belief State Updater (belief_state_updater.py)

V. EMV UTILITY FRAMEWORK

5.1 EMV Calculator (emv_calculator.py)

5.2 Utility Functions (utility_function.py)

5.3 Risk Model (risk_model.py)

VI. LEARNING SUBSYSTEM

6.1 Learning Update Engine (learning_update_engine.py)

6.2 Failure Analysis Module (failure_analysis_module.py)

VII. GOVERNANCE FRAMEWORK

7.1 Agent Constitution (agent_constitution.md)

7.2 Constraint Enforcement (constraint_enforcement_engine.py)

7.3 Decision Trace Log (decision_trace_log.jsonl)

VIII. ELITE OPTIONAL MODULES

8.1 Multi-Agent Interface (multi_agent_interface.py)

8.2 Scenario Simulator (scenario_simulator.py)

8.3 Red Team Memory Partition (red_team_memory_partition.py)

APPENDICES

A. Complete File Manifest

B. Integration Guidelines

C. Security Considerations

I. Executive Overview

Document Purpose

This Technical Architecture Specification provides comprehensive documentation for the CARBON[6] Autonomous AI Agency framework—a production-grade infrastructure for deploying safe, aligned, and effective autonomous AI agents in enterprise environments.

The framework implements rigorous decision-theoretic principles through 24 interconnected modules spanning decision-making, memory management, environment modeling, value optimization, learning, and governance.

1.1 Purpose and Scope

The CARBON[6] Autonomous Agency framework addresses the fundamental challenge of deploying AI agents that can:

- **Operate Autonomously** — Make decisions and take actions without constant human oversight
- **Maintain Alignment** — Remain aligned with human values and organizational objectives
- **Ensure Safety** — Operate within defined boundaries with deterministic constraint enforcement
- **Optimize Value** — Maximize expected monetary value while managing risk appropriately
- **Learn Continuously** — Improve performance through experience while preserving alignment
- **Provide Transparency** — Maintain complete audit trails for all decisions and actions

1.2 Architectural Philosophy

The architecture is grounded in three foundational principles:

CONFIDENTIAL

Markov Decision Process (MDP) Foundation

All agent behavior is modeled as a formal MDP with clearly defined state spaces, action spaces, transition dynamics, and reward functions. This provides mathematical rigor and enables formal verification of agent properties.

Expected Monetary Value (EMV) Optimization

Decisions are made by computing expected monetary values across all possible outcomes, weighted by probability and transformed through appropriate utility functions. This ensures economically rational decision-making.

Hierarchical Constraint Architecture

A multi-layered constraint system enforces safety at every level—from hard constitutional constraints that can never be violated, to soft operational guidelines that can be overridden with appropriate authorization.

1.3 System Components Overview

Subsystem	Components	Primary Function
Core Decision	3 modules	MDP specification, policy definition, reward modeling
Memory Protocol	6 modules	Experience storage, encoding, retrieval, and lifecycle management
Environment	4 modules	State modeling, observation, belief updating
EMV Utility	3 modules	Value calculation, utility transformation, risk assessment
Learning	2 modules	Policy improvement, failure analysis
Governance	3 modules	Constitution, constraints, audit logging
Elite Optional	3 modules	Multi-agent coordination, simulation, adversarial testing

CONFIDENTIAL

II. Core Decision Subsystem

The Core Decision Subsystem provides the mathematical foundation for all agent decision-making. It implements a complete Markov Decision Process specification with policy definitions and reward modeling.

2.1 MDP Specification

```
core_decision/mdp_spec.yaml
```

The MDP specification defines the formal structure of the agent's decision problem. It serves as the "constitutional document" for agent behavior.

State Space Definition

The state space is partitioned into observable and latent components:

Observable States	Description	Type
task_status	Current task execution state	Categorical
resource_availability	Available computational and external resources	Continuous [0,1]
constraint_status	Current constraint satisfaction state	Boolean
time_remaining	Time until deadline or timeout	Continuous

Latent States	Description	Inference Method
user_intent	Inferred user objectives	Bayesian inference
environment_stability	Estimated environment volatility	Kalman filtering
task_complexity	Estimated difficulty of current task	Feature-based estimation

CONFIDENTIAL

Action Space

The action space defines all possible agent actions, organized hierarchically:

`OBSERVE`	Gather information from environment without modifying state
`ANALYZE`	Process and analyze available information
`RECOMMEND`	Generate recommendation for human approval
`EXECUTE`	Take autonomous action within authorized scope
`ESCALATE`	Escalate decision to higher authority
`DEFER`	Postpone action pending additional information

Transition Model

The transition model specifies how actions affect state:

```
P(s' | s, a) = Σ P(s' | s, a, θ) * P(θ | history)

Where:
  s'    = next state
  s     = current state
  a     = action taken
  θ     = environment parameters (learned from history)
```

Discount Factor

The discount factor $\gamma = 0.95$ balances immediate and future rewards, with dynamic adjustment based on urgency:

```
γ_effective = γ_base * (1 - urgency_factor)

Where urgency_factor ∈ [0, 0.3] based on deadline proximity
```

2.2 Policy Definition

```
core_decision/policy_definition.py
```


CONFIDENTIAL

The policy module defines how agents select actions given states. Multiple policy types are supported:

Policy Types

DeterministicPolicy

Maps states directly to actions via a lookup table or function. Used when optimal action is unambiguous.

```
π(s) = a*  where a* = argmax_a Q(s, a)
```

StochasticPolicy

Maps states to probability distributions over actions. Enables exploration and handles uncertainty.

```
π(a|s) = exp(Q(s,a)/τ) / Σ_a' exp(Q(s,a')/τ)

Where τ = temperature parameter controlling exploration
```

EpsilonGreedyPolicy

Balances exploitation and exploration with configurable epsilon decay.

```
π(a|s) = {
    1-ε+ε/|A|  if a = argmax Q(s,a)
    ε/|A|      otherwise
}

ε_t = max(ε_min, ε_0 * decay^t)
```

Policy Manager

The PolicyManager orchestrates policy selection and enforcement:

```
class PolicyManager:
    def select_action(self, state: State) -> Action:
        # 1. Pre-filter actions through constraint engine
        valid_actions = self.constraint_engine.filter(state, self.all_actions)

        # 2. Apply active policy to select from valid actions
        action = self.active_policy.select(state, valid_actions)

        # 3. Log decision for audit
        self.decision_log.record(state, action, self.active_policy)

        return action
```

2.3 Reward Model

```
core_decision/reward_model.py
```

The reward model defines the objective function that guides agent behavior.

EMV Reward Model

The primary reward model computes Expected Monetary Value:

```
R(s, a, s') = Σ_o P(o | s, a, s') * V(o)

Where:
  o = outcome
  P(o | s, a, s') = probability of outcome given transition
  V(o) = monetary value of outcome
```

Risk-Adjusted Reward Model

Incorporates risk preferences through utility transformation:

CONFIDENTIAL

```
R_adjusted(s, a) = U(EMV(s, a)) - λ * Risk(s, a)

Where:
  U() = utility function (e.g., CRRA)
  λ = risk aversion coefficient
  Risk() = risk measure (VaR, CVaR, variance)
```

Multi-Objective Reward

Supports multiple objectives with configurable weights:

```
R_total = Σ_i w_i * R_i(s, a)

Subject to: Σ w_i = 1, w_i ≥ 0
```

Objective	Weight Range	Description
Task Completion	0.3 - 0.5	Primary task success
Resource Efficiency	0.1 - 0.2	Minimize resource consumption
Safety Margin	0.2 - 0.3	Maintain safety buffer
User Satisfaction	0.1 - 0.2	Inferred user preference alignment

III. Enhanced Memory Protocol (EMP)

The Enhanced Memory Protocol provides a comprehensive system for experience storage, retrieval, and lifecycle management. It enables agents to learn from past interactions while maintaining appropriate information governance.

3.1 Memory Schema

```
memory_protocol/memory_schema.json
```

The memory schema defines the canonical structure for all memory entries:

Memory Types

Type	Description	Retention
`episodic`	Specific experiences and events	Medium-term, subject to decay
`semantic`	General knowledge and facts	Long-term, stable
`strategic`	Learned strategies and heuristics	Long-term, updateable
`constraint`	Hard constraints and boundaries	Permanent, immutable
`procedural`	Learned procedures and workflows	Long-term, refinable

CONFIDENTIAL

Memory Entry Structure

```
{
  "memory_id": "string (UUID)",
  "memory_type": "episodic | semantic | strategic | constraint | procedural",
  "content": {
    "raw_data": "any",
    "embedding": "float[]",
    "summary": "string"
  },
  "metadata": {
    "created_at": "ISO 8601 timestamp",
    "source": "string",
    "confidence": "float [0-1]",
    "salience": "float [0-1]",
    "access_count": "integer",
    "last_accessed": "ISO 8601 timestamp"
  },
  "decay_profile": {
    "decay_type": "exponential | power | linear | step",
    "half_life_hours": "float",
    "minimum_salience": "float"
  },
  "provenance": {
    "original_source": "string",
    "derivation_chain": "string[]",
    "verification_status": "verified | unverified | disputed"
  }
}
```

3.2 Memory Encoder

```
memory_protocol/memory_encoder.py
```

The memory encoder transforms raw experiences into structured memory entries:

Encoding Pipeline

1. **Input Normalization** — Standardize input format and structure
2. **Type Classification** — Determine appropriate memory type
3. **Content Extraction** — Extract key information from experience
4. **Embedding Generation** — Create vector representation
5. **Metadata Assignment** — Attach creation metadata and initial salience

6. **Decay Profile Selection** — Assign appropriate decay characteristics
7. **Validation** — Verify against schema before storage

Batch Encoding

The BatchMemoryEncoder provides efficient processing of multiple experiences:

```
class BatchMemoryEncoder:
    def encode_batch(
        self,
        experiences: List[RawExperience],
        parallel: bool = True
    ) -> List[MemoryEntry]:

        if parallel:
            with ThreadPoolExecutor(max_workers=4) as executor:
                return list(executor.map(self.encode_single, experiences))
        else:
            return [self.encode_single(exp) for exp in experiences]
```

3.3 Salience Engine

```
memory_protocol/memory_salience_engine.py
```

The salience engine computes and updates memory importance scores:

Salience Computation

```
S(m) = w₁·EMV_impact(m) + w₂·risk_factor(m) + w₃·recency(m) + w₄·frequency(m)

Where:
  EMV_impact(m) = normalized absolute impact on expected value
  risk_factor(m) = amplification for risk-related memories
  recency(m) = exponential decay based on time since creation
  frequency(m) = log(access_count + 1) / log(max_access + 1)
```


CONFIDENTIAL

Default Weights

Factor	Weight	Rationale
EMV Impact	0.35	Primary driver of decision relevance
Risk Factor	0.25	Safety-critical memories amplified
Recency	0.25	Recent experiences more relevant
Frequency	0.15	Frequently accessed = frequently useful

3.4 Retrieval Policy

```
memory_protocol/memory_retrieval_policy.py
```

The retrieval policy determines when and how memories are accessed:

Retrieval Triggers

Automatic Retrieval Triggers

- **Decision Uncertainty** — Confidence below threshold triggers relevant memory search
- **Risk Threshold Breach** — High-risk situation retrieves constraint and safety memories
- **Policy Conflict** — Conflicting options trigger strategic memory retrieval
- **Novel State Detection** — Unfamiliar state triggers broad memory search

CONFIDENTIAL

Policy Variants

Policy	Salience Threshold	Max Results	Use Case
Standard	0.3	10	Normal operations
Conservative	0.5	5	High-stakes decisions
Aggressive	0.1	25	Exploration, learning

3.5 Decay and Compression

```
memory_protocol/memory_decay_and_compression.py
```

Memory lifecycle management through decay and compression:

Decay Functions

```
Exponential:  S(t) = S₀ · e^(-λt)         [Default for episodic]
Power Law:    S(t) = S₀ · (1 + t)^(-α)    [For semantic memories]
Linear:       S(t) = max(S_min, S₀ - kt)  [For time-bounded tasks]
Step:         S(t) = S₀ if t < T else 0   [For deadline-based events]
```

Compression Strategy

When memory capacity approaches limits, compression is triggered:

1. **Identify Candidates** — Memories with salience below compression threshold
2. **Group by Type** — Cluster similar low-salience memories
3. **Generate Summary** — Create compressed representation preserving key insights
4. **Supersede Originals** — Replace individual memories with summary
5. **Update References** — Maintain provenance chain to originals

3.6 Immutable Memory Registry

```
memory_protocol/immutable_memory_registry.yaml
```


CONFIDENTIAL

IMMUTABLE CONSTRAINTS

The following memory categories cannot be modified, overridden, or deleted. They persist indefinitely and supersede all other memories in conflict resolution.

Categories

Category	Example Entries	Purpose
Hard Constraints	No physical harm, no privacy violations	Absolute behavioral boundaries
Safety Rules	Fail-safe defaults, escalation requirements	Operational safety
Governance Mandates	Audit requirements, authorization levels	Compliance and oversight
Ethical Boundaries	Truthfulness, beneficence, autonomy respect	Value alignment

IV. Environment Modeling

The Environment subsystem provides the agent's interface with the external world, managing observation, state estimation, and belief updates under uncertainty.

4.1 Environment State Model

```
environment/environment_state_model.py
```

State Variable Types

Type	Values	Example
Continuous	Real numbers in range	confidence_level ∈ [0, 1]
Discrete	Finite set	task_status ∈ {pending, active, complete}
Binary	{true, false}	constraint_satisfied
Categorical	Unordered set	user_preference ∈ {speed, quality, cost}

Partial Observability

The PartiallyObservableStateModel maintains belief distributions over hidden states:

```
class PartiallyObservableStateModel:
    def __init__(self):
        self.belief_state: Dict[str, Distribution] = {}

    def update_belief(self, observation: Observation) -> None:
        for var in self.hidden_variables:
            likelihood = self.observation_model.likelihood(observation, var)
            self.belief_state[var] = self._bayesian_update(
                prior=self.belief_state[var],
                likelihood=likelihood
            )
```


CONFIDENTIAL

4.2 Variable Registry

```
environment/environment_variable_registry.yaml
```

Complete enumeration of environment variables:

Task Variables

- `task_id` — Unique task identifier
- `task_type` — Classification of task
- `task_priority` — Urgency level [1-5]
- `task_deadline` — Completion deadline (optional)
- `task_progress` — Completion percentage [0-100]

Agent Variables

- `confidence_level` — Current decision confidence [0-1]
- `risk_exposure` — Aggregate risk level [0-1]
- `resource_utilization` — Current resource usage [0-1]
- `constraint_violations` — Count of active violations

Derived Variables

- `urgency_score` — Computed from deadline proximity
- `complexity_estimate` — Inferred task difficulty
- `success_probability` — Estimated likelihood of success

4.3 Environment Observer

```
environment/environment_observer.py
```

Observation Sources

Source	Reliability	Latency
Internal State	1.0	Immediate
User Input	0.9	Variable
API Response	0.95	Network-dependent
Sensor Data	0.8	Polling interval
Inference	Variable	Computation time

Observation Processing

```
def process_observation(self, raw_obs: RawObservation) -> ProcessedObservation:
    # 1. Validate source
    if not self.validate_source(raw_obs.source):
        raise UntrustedSourceError(raw_obs.source)

    # 2. Normalize values
    normalized = self.normalize(raw_obs.values)

    # 3. Handle missing data
    imputed = self.impute_missing(normalized)

    # 4. Apply reliability weighting
    weighted = self.apply_reliability(imputed, raw_obs.source)

    # 5. Timestamp and return
    return ProcessedObservation(
        values=weighted,
        timestamp=datetime.now(),
        source=raw_obs.source,
        confidence=self.compute_confidence(weighted)
    )
```

4.4 Belief State Updater

`environment/belief_state_updater.py`

CONFIDENTIAL

Update Methods

Bayesian Belief Updater

```
P(θ|D) ∝ P(D|θ) · P(θ)

posterior = likelihood × prior / evidence
```

Used for: Discrete states, categorical variables, hypothesis testing

Kalman Belief Updater

```
x̂□ = x̂□-1 + K□(z□ - Hx̂□-1)
K□ = P□-1Hᵀ(HP□-1Hᵀ + R)⁻¹
```

Used for: Continuous states, linear dynamics, real-valued estimates

Exponential Moving Average

```
x̂□ = α · x□ + (1-α) · x̂□-1
```

Used for: Trend tracking, simple smoothing, high-frequency updates

CONFIDENTIAL

V. EMV Utility Framework

The EMV Utility Framework provides the economic reasoning capability for the agent, enabling value-based decision-making with appropriate risk management.

5.1 EMV Calculator

```
emv_utility/emv_calculator.py
```

Core Calculation

```
EMV(a) = Σᵢ P(oᵢ | a) × V(oᵢ)

Where:
  a = action under consideration
  oᵢ = possible outcome i
  P(oᵢ | a) = probability of outcome given action
  V(oᵢ) = monetary value of outcome
```

Outcome Enumeration

The OutcomeEnumerator generates the probability-weighted outcome set:

```
class OutcomeEnumerator:
    def enumerate(
        self,
        state: State,
        action: Action
    ) -> List[WeightedOutcome]:

        # Generate base outcomes from transition model
        base_outcomes = self.transition_model.predict_outcomes(state, action)

        # Expand with uncertainty
        expanded = self.expand_uncertainty(base_outcomes)

        # Normalize probabilities
        total_prob = sum(o.probability for o in expanded)
        for outcome in expanded:
            outcome.probability /= total_prob

        return expanded
```

Action Comparison

```
def compare_actions(
    self,
    state: State,
    actions: List[Action]
) -> ActionRanking:

    emv_scores = {}
    for action in actions:
        emv_scores[action] = self.calculate_emv(state, action)

    ranked = sorted(
        emv_scores.items(),
        key=lambda x: x[1],
        reverse=True
    )

    return ActionRanking(
        best_action=ranked[0][0],
        scores=emv_scores,
        margin=ranked[0][1] - ranked[1][1] if len(ranked) > 1 else float('inf')
    )
```

5.2 Utility Functions

```
emv_utility/utility_function.py
```

Utility functions transform monetary values to account for risk preferences:

Available Functions

Function	Formula	Risk Attitude
Linear	$U(x) = x$	Risk Neutral
Logarithmic	$U(x) = \ln(x + 1)$	Risk Averse (decreasing)
Exponential	$U(x) = 1 - e^{(-\alpha x)}$	Risk Averse (constant)
Power (CRRA)	$U(x) = x^{(1-\rho)} / (1-\rho)$	Risk Averse (configurable)
Prospect Theory	$U(x) = x^{\alpha}$ if $x \geq 0$, $-\lambda(-x)^{\beta}$ if $x<0$	Loss Averse

Adaptive Utility

The AdaptiveUtilityFunction adjusts risk aversion based on context:

```
class AdaptiveUtilityFunction:
    def compute(self, value: float, context: Context) -> float:
        # Increase risk aversion when:
        # - Stakes are high
        # - Resources are low
        # - Time pressure is high

        risk_aversion = self.base_risk_aversion

        if context.stakes > self.high_stakes_threshold:
            risk_aversion *= 1.5

        if context.resource_availability < 0.3:
            risk_aversion *= 1.3

        return self.power_utility(value, risk_aversion)
```

5.3 Risk Model

```
emv_utility/risk_model.py
```

Risk Measures

Value at Risk (VaR)

The maximum loss not exceeded with a given confidence level.

```
VaR_α = -inf{x : P(X ≤ x) ≥ α}
```

Example: VaR_{95} = $10,000 means 95% confident loss won't exceed $10,000

Conditional Value at Risk (CVaR)

Expected loss in the worst α% of cases.

```
CVaR_α = E[X | X ≤ VaR_α]
```

More conservative than VaR—captures tail risk.

Risk Categories

Category	Examples	Mitigation Focus
Operational	System failure, timeout, resource exhaustion	Redundancy, monitoring
Financial	Cost overrun, opportunity cost, market volatility	Budgets, hedging
Compliance	Policy violation, regulatory breach	Constraints, auditing
Reputational	User dissatisfaction, trust erosion	Quality controls, escalation
Strategic	Goal misalignment, suboptimal outcomes	Alignment checks, oversight

CONFIDENTIAL

VI. Learning Subsystem

The Learning subsystem enables the agent to improve performance through experience while maintaining alignment and safety guarantees.

6.1 Learning Update Engine

```
learning/learning_update_engine.py
```

Learning Algorithms

Temporal Difference Learning

```
Q(s,a) ← Q(s,a) + α[r + γ·max_a' Q(s',a') - Q(s,a)]

Where:
  α = learning rate
  γ = discount factor
  r = immediate reward
  s' = next state
```

Policy Gradient

```
∇J(θ) = E[∇log π_θ(a|s) · A(s,a)]

Where:
  θ = policy parameters
  π_θ = parameterized policy
  A(s,a) = advantage function
```


CONFIDENTIAL

Salience-Based Learning

Learning rates are modulated by experience salience:

```
α_effective = α_base × salience(experience)

# High-salience experiences (critical outcomes, errors)
# have stronger learning effect
```

Experience Replay

```
class ExperienceReplay:
    def __init__(self, capacity: int = 10000):
        self.buffer = deque(maxlen=capacity)

    def sample_batch(self, batch_size: int) -> List[Experience]:
        # Priority sampling based on TD error
        priorities = [self.compute_priority(exp) for exp in self.buffer]
        probs = np.array(priorities) / sum(priorities)

        indices = np.random.choice(
            len(self.buffer),
            size=batch_size,
            p=probs
        )

        return [self.buffer[i] for i in indices]
```

6.2 Failure Analysis Module

```
learning/failure_analysis_module.py
```

Systematic analysis of failures to prevent recurrence:

Root Cause Analysis

```
class FailureAnalysisModule:
    def analyze_failure(
        self,
        failure_event: FailureEvent
    ) -> FailureAnalysis:

        # 1. Extract decision trace
        trace = self.get_decision_trace(failure_event.session_id)

        # 2. Identify contributing factors
        factors = self.identify_factors(trace, failure_event)

        # 3. Determine root cause
        root_cause = self.find_root_cause(factors)

        # 4. Generate recommendations
        recommendations = self.generate_recommendations(root_cause)

        # 5. Update policy penalties
        self.apply_policy_penalties(root_cause, failure_event.severity)

        return FailureAnalysis(
            failure=failure_event,
            factors=factors,
            root_cause=root_cause,
            recommendations=recommendations
        )
```

Failure Categories

Category	Typical Causes	Policy Response
Information Failure	Missing data, stale information	Increase observation before action
Estimation Error	Probability miscalibration	Widen confidence intervals
Execution Failure	Action failed to achieve intended effect	Add verification steps
Constraint Violation	Boundary exceeded	Strengthen constraint checking
Goal Misalignment	Achieved stated goal but not intent	Improve intent modeling

VII. Governance Framework

The Governance Framework ensures safe, aligned, and auditable agent operation through constitutional constraints, enforcement mechanisms, and comprehensive logging.

7.1 Agent Constitution

```
governance/agent_constitution.md
```

The Agent Constitution establishes the foundational principles and boundaries for all agent behavior.

Mission Hierarchy

1. **Safety** — Prevent harm to humans, systems, and data
2. **Compliance** — Adhere to all applicable laws and policies
3. **Effectiveness** — Achieve designated objectives efficiently
4. **Transparency** — Maintain auditability and explainability
5. **Improvement** — Continuously learn and enhance capabilities

CONFIDENTIAL

Absolute Prohibitions

HARD CONSTRAINTS - Cannot Be Overridden

1. Cause or facilitate physical harm to humans
2. Expose personal identifiable information without consent
3. Misrepresent AI nature when directly asked
4. Bypass or disable safety mechanisms
5. Assist in creating weapons, malware, or harmful substances
6. Take actions beyond defined authorization levels
7. Permanently delete data without explicit confirmation
8. Operate in ways that evade logging or monitoring

Authority Levels

1	**Observation** Read data, gather information — no state modification
2	**Analysis** Process, analyze, generate insights from available data
3	**Recommendation** Suggest actions for human approval
4	**Autonomous** Execute pre-approved action types independently
5	**Supervisory** Oversee other agents (with human oversight)

7.2 Constraint Enforcement Engine

```
governance/constraint_enforcement_engine.py
```

Constraint Types

Type	Override Possible	Violation Response
HARD	Never	Block action, log, alert
SOFT	With authorization	Warn, request confirmation
ADVISORY	Always	Log warning only

Enforcement Flow

```
def check_constraints(self, state: State, action: Action) -> EnforcementResult:
    violations = []

    # Check constraints in priority order
    for constraint in sorted(self.constraints, key=lambda c: c.priority, reverse=True):
        if constraint.is_violated(state, action):
            violations.append(self.create_violation(constraint, state, action))

            if constraint.type == ConstraintType.HARD:
                # Hard constraint - block immediately
                return EnforcementResult(
                    allowed=False,
                    violations=violations,
                    action=EnforcementAction.BLOCK
                )

    # All hard constraints passed
    return EnforcementResult(
        allowed=True,
        violations=violations,
        warnings=[v.message for v in violations]
    )
```


CONFIDENTIAL

Escalation Protocol

Level	Trigger	Response
1	Advisory violation	Log and continue
2	Soft constraint violation	Warn and request confirmation
3	Hard constraint violation	Block action, escalate to supervisor
4	Critical safety violation	Immediate shutdown, alert all principals

7.3 Decision Trace Log

```
governance/decision_trace_log.jsonl
```

Complete audit trail for all agent decisions in JSON Lines format:

Log Entry Schema

```
{
  "trace_id": "uuid",
  "timestamp": "ISO 8601",
  "session_id": "string",
  "agent_id": "string",
  "decision": {
    "state_snapshot": {...},
    "action_considered": [...],
    "action_selected": "string",
    "emv_scores": {...},
    "policy_used": "string"
  },
  "constraints": {
    "checked": [...],
    "violations": [...],
    "overrides_applied": [...]
  },
  "outcome": {
    "result": "success | failure | partial",
    "actual_value": number,
    "expected_value": number,
    "variance_explanation": "string"
  }
}
```


CONFIDENTIAL

Retention Policy

- **Real-time logs** — 30 days hot storage
- **Aggregated logs** — 1 year warm storage
- **Critical events** — Indefinite cold storage
- **Constraint violations** — 7 years (compliance)

VIII. Elite Optional Modules

Advanced capabilities for enterprise deployments requiring multi-agent coordination, scenario simulation, and adversarial testing.

8.1 Multi-Agent Interface

```
elite_optional/multi_agent_interface.py
```

Coordination Protocols

Protocol	Use Case	Characteristics
Hierarchical	Clear authority structure	Coordinator assigns tasks
Democratic	Peer decision-making	Voting on proposals
Auction	Resource allocation	Agents bid for tasks
Contract Net	Flexible task assignment	Announce, bid, award cycle
Blackboard	Collaborative problem-solving	Shared workspace

Shared Memory

```
class SharedMemory:
    async def put(
        self,
        key: str,
        value: Any,
        owner_id: str,
        ttl_seconds: Optional[int] = None,
        access_list: Optional[Set[str]] = None
    ) -> bool:
        """
        Store value in shared memory with:
        - Owner tracking
        - Optional TTL
        - Optional access control list
        """

    async def get(
        self,
        key: str,
        requester_id: str
    ) -> Optional[Any]:
        """
        Retrieve value if:
        - Key exists
        - Not expired
        - Requester has access
        """
```

Negotiation Protocol

```
async def initiate_negotiation(
    self,
    initiator_id: str,
    participants: Set[str],
    topic: str,
    initial_offer: Dict,
    deadline_seconds: int = 60
) -> str:
    """
    Start multi-party negotiation:
    1. Create negotiation state
    2. Notify all participants
    3. Accept offers until deadline
    4. Determine outcome
    """
```

8.2 Scenario Simulator

```
elite_optional/scenario_simulator.py
```

Simulation Types

Monte Carlo Simulation

Samples from variable distributions to estimate outcome distributions.

- Default: 1,000 samples
- Convergence detection
- Percentile reporting (5th, 25th, 50th, 75th, 95th)

Scenario Analysis

Evaluates specific scenarios:

- Baseline (expected values)
- Optimistic (90th percentile)
- Pessimistic (10th percentile)
- Stress tests (extreme values)

Counterfactual Analysis

Answers: "What would have happened if we had done X instead?"

```
Regret = max(0, E[V(alt_action)] - V(actual_action))
```

Foresight Planning

Multi-step lookahead over planning horizon:

```
def run_foresight(
    self,
    variables: List[ScenarioVariable],
    actions: List[str],
    horizon_steps: int = 5
) -> Dict[str, List[SimulationResult]]:
    """
    For each action, simulate outcomes over horizon,
    evolving variables at each step.
    """
```

8.3 Red Team Memory Partition

`elite_optional/red_team_memory_partition.py`

Attack Types

Attack	Description	Severity
Prompt Injection	Attempt to override system instructions	High
Memory Poisoning	Inject false memories	High
Constraint Bypass	Gradual escalation past boundaries	Critical
Reward Hacking	Game metrics without achieving goals	Medium
Information Leak	Extract sensitive system information	High

Isolation Levels

Level	Read Main Memory	Write Main Memory	Use Case
NONE	✓	✓	Not recommended
SOFT	✓	✗	Read-only testing
HARD	✗	✗	Full isolation
SNAPSHOT	Copy	To copy	Safe experimentation

Test Execution

```
def run_test_suite(
    self,
    agent_handler: Callable,
    attack_types: Optional[List[AttackType]] = None
) -> Dict[str, Any]:
    """
    Execute adversarial test suite:
    - Generate scenarios for each attack type
    - Run attacks in isolation
    - Record defenses triggered
    - Generate vulnerability report
    - Calculate security score
    """

    return {
        "passed": count,
        "failed": count,
        "pass_rate": float,
        "vulnerabilities": List[VulnerabilityReport],
        "security_score": float  # 0-100
    }
```

Appendix A: Complete File Manifest

#	File Path	Type	Purpose
1	core_decision/mdp_spec.yaml	YAML	MDP constitutional specification
2	core_decision/policy_definition.py	Python	Policy types and manager
3	core_decision/reward_model.py	Python	Reward computation models
4	memory_protocol/memory_schema.json	JSON	Memory entry schema
5	memory_protocol/memory_encoder.py	Python	Experience to memory transformation
6	memory_protocol/memory_salience_engine.py	Python	Importance scoring
7	memory_protocol/memory_retrieval_policy.py	Python	When and how to retrieve
8	memory_protocol/ memory_decay_and_compression.py	Python	Lifecycle management
9	memory_protocol/ immutable_memory_registry.yaml	YAML	Permanent constraints
10	environment/environment_state_model.py	Python	State representation
11	environment/ environment_variable_registry.yaml	YAML	Variable definitions
12	environment/environment_observer.py	Python	Observation processing
13	environment/belief_state_updater.py	Python	Belief maintenance
14	emv_utility/emv_calculator.py	Python	Expected value computation
15	emv_utility/utility_function.py	Python	Utility transformations
16	emv_utility/risk_model.py	Python	Risk assessment
17	learning/learning_update_engine.py	Python	Policy improvement
18	learning/failure_analysis_module.py	Python	Failure investigation

CONFIDENTIAL

19	governance/agent_constitution.md	Markdown	Foundational charter
20	governance/constraint_enforcement_engine.py	Python	Constraint checking
21	governance/decision_trace_log.jsonl	JSONL	Audit trail
22	elite_optional/multi_agent_interface.py	Python	Agent coordination
23	elite_optional/scenario_simulator.py	Python	What-if analysis
24	elite_optional/red_team_memory_partition.py	Python	Security testing

Appendix B: Integration Guidelines

B.1 Quick Start

```
# 1. Initialize core components
from core_decision.policy_definition import create_policy_manager
from core_decision.reward_model import create_emv_reward_model
from governance.constraint_enforcement_engine import create_constraint_engine

policy_manager = create_policy_manager()
reward_model = create_emv_reward_model()
constraint_engine = create_constraint_engine()

# 2. Initialize memory system
from memory_protocol.memory_encoder import create_memory_encoder
from memory_protocol.memory_salience_engine import create_salience_engine

encoder = create_memory_encoder()
salience_engine = create_salience_engine()

# 3. Initialize environment
from environment.environment_observer import create_environment_observer
from environment.belief_state_updater import create_belief_updater

observer = create_environment_observer()
belief_updater = create_belief_updater()

# 4. Create agent
agent = AutonomousAgent(
    policy_manager=policy_manager,
    reward_model=reward_model,
    constraint_engine=constraint_engine,
    memory_encoder=encoder,
    salience_engine=salience_engine,
    observer=observer,
    belief_updater=belief_updater
)
```

B.2 Configuration Best Practices

- **Start Conservative** — Begin with low authority levels and strict constraints
- **Monitor Closely** — Review decision traces during initial deployment

CONFIDENTIAL

- **Tune Incrementally** — Adjust risk parameters based on observed behavior
- **Test Adversarially** — Use red team module before production
- **Audit Regularly** — Review constraint violations and escalations

B.3 Deployment Checklist

Phase	Requirements	Sign-off
Development	All unit tests passing, code review complete	☐
Security	Red team test suite passed (>90% score)	☐
Compliance	Constitution reviewed, constraints validated	☐
Integration	End-to-end testing complete	☐
Monitoring	Logging, alerting, dashboards configured	☐
Rollback	Rollback procedure documented and tested	☐

Appendix C: Security Considerations

C.1 Threat Model

Threat	Mitigation	Detection
Prompt Injection	Input sanitization, structured prompts	Pattern matching, anomaly detection
Memory Tampering	Integrity checks, signed memories	Hash verification, provenance audit
Privilege Escalation	Authority level enforcement	Action logging, boundary alerts
Data Exfiltration	Output filtering, classification	Content scanning, rate limiting
Denial of Service	Resource limits, rate limiting	Usage monitoring, threshold alerts

C.2 Security Hardening

- **Principle of Least Privilege** — Grant minimum necessary authority
- **Defense in Depth** — Multiple layers of constraint enforcement
- **Fail Secure** — Default to safe state on errors
- **Complete Mediation** — Check constraints on every action
- **Separation of Duties** — Different agents for different functions

C.3 Incident Response

1. **Detection** — Automated alerts on constraint violations
2. **Containment** — Automatic escalation and potential shutdown
3. **Investigation** — Decision trace analysis
4. **Remediation** — Constraint updates, policy adjustments
5. **Recovery** — Controlled restart with enhanced monitoring

CONFIDENTIAL

6. **Lessons Learned** — Update failure analysis module

CARBON[6] Autonomous AI Agency Framework
Technical Architecture Specification v1.0.0
© 2026 Carbon Development Team. All Rights Reserved.

This document contains proprietary and confidential information.
Unauthorized distribution or reproduction is prohibited.